

Методические рекомендации
по выполнению индивидуального проекта по дисциплине
"Интеллектуальные системы и программные продукты"

Цель проекта: Разработать прототип программного продукта , интегрирующего интеллектуальный модуль на основе методов искусственного интеллекта, и продемонстрировать его работоспособность .

Поэтапный план выполнения проекта

Этап 1: Определение и анализ предметной области

1. Выберите конкретную и узкую предметную область (например, "Прогнозирование продаж в розничной сети", "Классификация изображений сорняков и культурных растений", "Чат-бот с NLP для поддержки клиентов интернет-магазина", "Система рекомендации фильмов").

2. Опишите проблему в этой области, которую можно решить с помощью ИИ. Ответьте на вопросы:

В чем заключается основная задача?

Кто потенциальные пользователи системы?

Каковы текущие способы решения этой проблемы (без ИИ)?

Какие преимущества даст внедрение интеллектуальной системы?

Этап 2: Постановка цели и задач проекта

1. Сформулируйте одну общую цель проекта.

Например: "Разработать прототип системы для прогнозирования энергопотребления здания на основе исторических данных и прогноза погоды".

2. Разбейте цель на конкретные, измеримые задачи.

Пример:

1. Проанализировать предметную область и определить требования к данным.
2. Выбрать и обосновать технологию ИИ для решения задачи.
3. Собрать или сгенерировать набор данных.
4. Обучить и валидировать модель машинного обучения.
5. Разработать архитектуру программного продукта.
6. Реализовать прототип с базовым интерфейсом.
7. Провести тестирование и подготовить презентацию для защиты.

Этап 3: Определение необходимых данных и требований к ним

1. Укажите, откуда будут браться данные (публичные датасеты Kaggle, UCI ML Repository, API, синтетические данные, парсинг с разрешения).

2. Опишите, какими характеристиками должны обладать данные:

Тип данных: табличные, текстовые, изображения, временные ряды.

Объем: минимальное количество записей для обучения.

Качество: требования к отсутствию пропусков, выбросов, несбалансированности классов.

Признаки: какие поля (features) необходимы для решения задачи.

Этап 4: Выбор и обоснование технологии ИИ

1. Рассмотрите 2-3 подходящие технологии (например, для классификации: Решающие деревья, Случайный лес, Нейронная сеть, продукционная система). Кратко опишите принцип работы каждой.

2. Сравните их по критериям: точность, интерпретируемость, скорость обучения, требования к вычислительным ресурсам, простота реализации.

3. Выберите одну технологию и подробно объясните, почему она лучше всего подходит для вашей конкретной задачи и контекста.

Этап 5: Обучение и валидация модели

1. Предобработка данных - Опишите проведенные операции: кодирование категориальных переменных, нормализация, обработка пропусков, augmentation (для изображений).

2. Обучение модели: Опишите процесс обучения, используемые библиотеки (Scikit-learn, TensorFlow, PyTorch), параметры модели.

3. Валидация и оценка:

Для моделей ML: Приведите метрики качества (Accuracy, Precision, Recall, F1-Score, MSE, R^2) на тестовой выборке. Постройте матрицу ошибок (Confusion Matrix) или графики.

Для других систем (например, экспертных): Проверьте работу системы на нескольких заранее подготовленных сценариях (кейсах), опишите входные данные и полученные выходные решения/рекомендации.

Этап 6: Проектирование программного продукта (UML)

1. Диаграмма вариантов использования (Use Case Diagram).

Покажите взаимодействие акторов (пользователь, администратор) с системой.

2. Диаграмма классов (Class Diagram)

Представьте основные классы вашей интеллектуальной системы.

Выделите отдельный класс для вашей ИИ-модели (например, PredictionModel).

Покажите его основные методы (.train(), .predict(), .save_model()) и атрибуты.

Отразите взаимосвязи этого класса с другими (например, с DataProcessor, UserInterface).

3. Диаграмма деятельности (Activity Diagram)

Детализируйте ключевой функционал, например, "Процесс получения прогноза" или "Взаимодействие агентов в системе".

Покажите поток данных и управления.

Этап 7: Описание модулей и интерфейсов и API

1. Описание программных модулей

2. Внутренние интерфейсы

Опишите, как интеллектуальный модуль взаимодействует с другими частями программы (например, как основной код передает данные в модель и получает предсказание).

3. Внешние интерфейсы (при необходимости)

Если используется внешний AI-as-a-Service (например, OpenAI API, Google Vision API), опишите, как происходит вызов.

Если ваш прототип сам предоставляет API, опишите его конечные точки (endpoints), форматы запросов и ответов (JSON).

Этап 8: Разработка прототипа

1. Создайте работающий прототип на выбранном языке программирования (Python, Java, C# и т.д.).

2. Интегрируйте обученную модель в программный продукт.

3. Реализуйте минимальный (базовый) пользовательский интерфейс (консольный, веб-интерфейс на Flask/Streamlit, или десктопное приложение), позволяющий продемонстрировать работу интеллектуальной системы.

Этап 9: Тестирование и демонстрация

1. Проведите модульное тестирование ключевых функций и интеграционное тестирование всего прототипа.

2. Подготовьте четкий сценарий для защиты: "Пользователь вводит данные X -> система обрабатывает -> выдает результат Y, который соответствует ожиданиям".

Этап 10: Оформление отчета и подготовка к защите

Требования к отчету (документ MS Word)

1. Титульный лист: (Вуз, кафедра, дисциплина, тема проекта, ФИО студента и преподавателя, год).
2. Оглавление.
3. Введение: Актуальность, проблема, цель и задачи проекта.
4. Глава 1. Анализ предметной области и постановка задачи.
5. Глава 2. Выбор и обоснование технологии ИИ.
6. Глава 3. Проектирование интеллектуальной системы: Разделы с UML-диаграммами и их описанием.
7. Глава 4. Реализация и тестирование прототипа: Описание среды разработки, библиотек, интерфейса, приведение примеров кода (только ключевых фрагментов), результатов тестирования.
8. Заключение: Выводы по работе, оценка достижения цели, перспективы развития.
9. Список использованных источников.
10. Приложения: (Исходный код, полные датасеты, конфигурационные файлы – по необходимости).

Форматирование: Шрифт Times New Roman, 14pt, межстрочный интервал 1.5, все поля 2 см.

Шаблон презентации для защиты (10-12 слайдов)

Слайд 1: Титульный. Тема проекта, ФИО студента, группа, ФИО научного руководителя.

Слайд 2: Актуальность и проблема. Почему эта задача важна? Какую проблему она решает?

Слайд 3: Цель и задачи проекта. Четкое формулирование цели и перечень задач.

Слайд 4: Анализ предметной области. Краткое описание выбранной области и анализ данных.

Слайд 5: Выбор технологии ИИ. Название выбранного метода/алгоритма, его краткое описание и обоснование выбора.

Слайд 6: Результаты обучения модели. Показать графики обучения, ключевые метрики качества (выделить лучшие результаты).

Слайд 7: Архитектура системы (UML). Показать самую наглядную диаграмму (например, Диаграмму классов или Диаграмму деятельности) и кратко ее пояснить.

Слайд 8: Прототип системы. Скриншот интерфейса вашего прототипа. Показать, как он выглядит.

Слайд 9: Демонстрация работы. Показать на примере: "Были входные данные X -> на выходе получили Y". Лучше в динамике (прямо во время защиты).

Слайд 10: Тестирование. Кратко описать, как тестировали систему, и какие результаты получили.

Слайд 11: Выводы. Подвести итоги: достигнута ли цель, каковы основные результаты, в чем заключается научная/практическая ценность работы.

Слайд 12: Спасибо за внимание! Вопросы?

Пример выполнения индивидуального проекта: "IntelliCourse: Интеллектуальная система рекомендации дополнительных дисциплин для студентов"

1. Определение и анализ предметной области

Предметная область: Образовательный процесс в университете, планирование индивидуальных траекторий обучения.

Проблема: Студенты часто выбирают дополнительные дисциплины (факультативы, курсы по выбору) хаотично, без учета своих карьерных целей, сильных сторон и целостности формируемых компетенций. Это приводит к снижению эффективности обучения и неоптимальному использованию образовательного потенциала.

Пользователи:

Основные: Студенты бакалавриата и магистратуры.

Косвенные: Деканаты и академические советники.

Текущее решение: Просмотр бумажного каталога курсов или электронной таблицы, консультации с руководителем, выбор по совету друзей.

Преимущества ИИ-системы: Персонализированные рекомендации, учитывающие историю успеваемости, интересы студента и требования рынка труда; снижение нагрузки на деканат; повышение удовлетворенности студентов.

2. Постановка цели и задач проекта

Цель: Разработать прототип интеллектуальной системы, предоставляющей студентам персонализированные рекомендации по выбору дополнительных дисциплин на основе анализа их академической успеваемости и заявленных интересов.

Задачи:

1. Проанализировать и формализовать требования к данным об успеваемости, курсах и студентах.
2. Выбрать и обосновать алгоритм рекомендательной системы.
3. Сгенерировать синтетический набор данных для обучения и тестирования.
4. Реализовать и обучить модель рекомендаций.
5. Спроектировать архитектуру системы с использованием UML.
6. Разработать веб-прототип с базовым интерфейсом для студента и деканата.
7. Провести тестирование на синтетических данных и подготовить отчет.

3. Определение необходимых данных и требований к данным

Необходимые данные:

1. **Данные о студентах:** ID, направление подготовки, история оценок по пройденным курсам.
2. **Данные о курсах:** ID, название, аннотация, направленность (теги, например: "Машинное обучение", "Веб-разработка", "Бизнес-аналитика"), академические требования.
3. **Данные о взаимодействиях:** Неявный feedback (факт прохождения курса) и явный feedback (рейтинги курсов от студентов).

Требования к данным:

- **Объем:** ~1000 студентов, ~100 курсов, ~10 000 записей об успеваемости.
- **Качество:** Отсутствие пропусков в обязательных полях (ID, оценки). Сбалансированность тегов курсов.
- **Источник:** На этапе прототипа данные будут синтезированы. В реальности - интеграция с АИС "Университет".

4. Выбор технологии искусственного интеллекта (обзор и обоснование)

Обзор технологий:

1. **Коллаборативная фильтрация (Collaborative Filtering):** Рекомендует курсы, которые понравились похожим студентам. Сильно зависит от данных о взаимодействиях.
2. **Content-Based фильтрация:** Рекомендует курсы, похожие на те, которые студент уже успешно прошел, по содержанию (тегам). Не зависит от данных по другим студентам.
3. **Гибридный подход:** Комбинирует оба метода для повышения точности и устранения "холодного старта" (проблемы для новых курсов или студентов).

Обоснование выбора: Выбран **Гибридный подход**. Для нового студента система будет полагаться на Content-Based подход (похожие курсы на основе его направленности). По мере накопления данных будет подключаться коллаборативная фильтрация для более точных рекомендаций. В качестве алгоритма используется **SVD (Singular Value Decomposition)** для коллаборативной части и **косинусное сходство** для content-based части. Реализация - библиотека scikit-surprise и scikit-learn.

5. Обучение и валидация модели

1. Предобработка:

Векторизация аннотаций курсов с помощью TF-IDF.

Создание матрицы "студент-курс" с оценками (явными или неявными).

Нормализация оценок.

2. Обучение:

Разделение данных на обучающую и тестовую выборку (80/20).

Обучение SVD модели на матрице взаимодействий.

Расчет матрицы косинусной схожести между курсами на основе TF-IDF векторов.

3. Валидация:

Метрики для SVD: RMSE (Root Mean Square Error) = 0.85 на тестовой выборке.

Метрики для рекомендаций (Precision@k, Recall@k): Для k=5: Precision@5 = 0.65, Recall@5 = 0.42. Это означает, что в 65% случаев рекомендованный курс был бы релевантен студенту.

6. Проектирование программного продукта (UML)

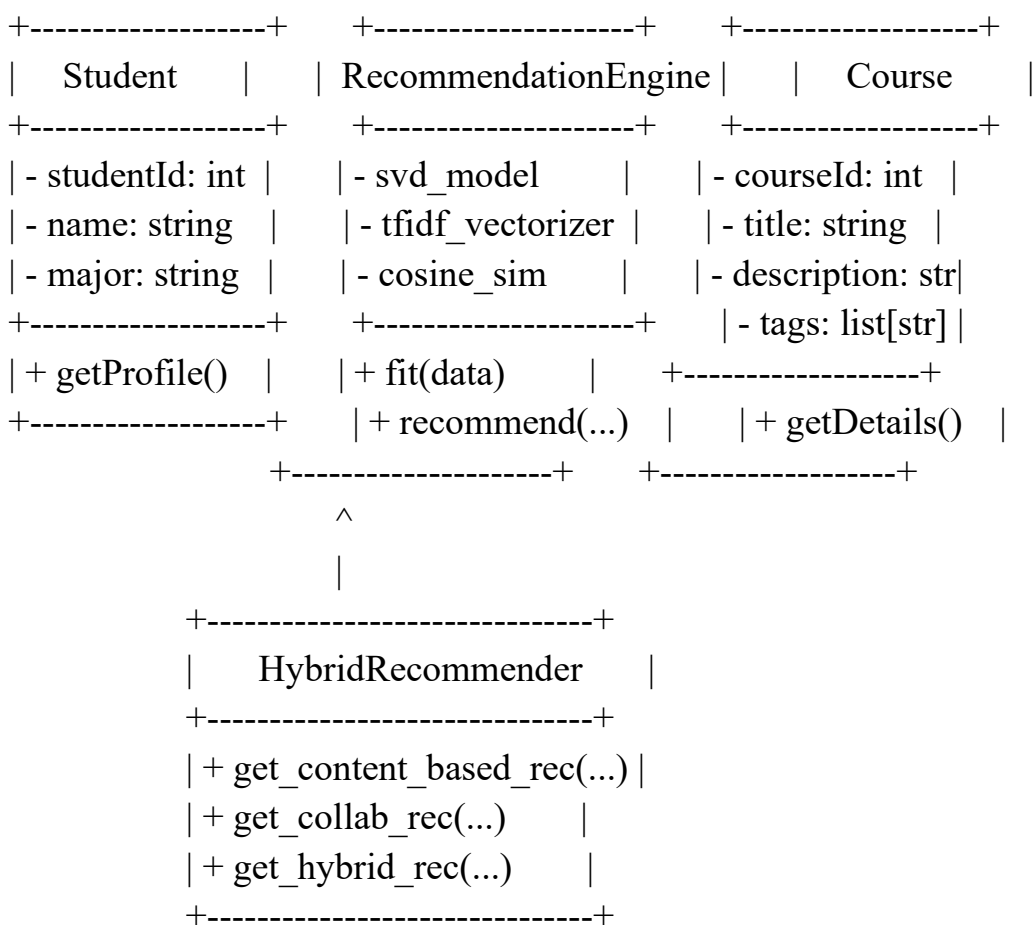
6.1. Диаграмма вариантов использования (Use Case Diagram):

Актор "Студент": Просмотр доступных курсов, получение рекомендаций, просмотр своего профиля.

Актор "Сотрудник деканата": Управление каталогом курсов (добавление, редактирование), просмотр агрегированной статистики по выбору курсов.

6.2. Диаграмма классов (Class Diagram):

text



- Класс RecommendationEngine инкапсулирует всю логику ИИ. Агрегирует в себе гибридную модель.

6.3. Диаграмма деятельности (Activity Diagram) для сценария "Получить рекомендации":

(Начало) -> [Студент заходит в систему] -> [Система загружает профиль студента и его историю] ->

[Вызов RecommendationEngine] -> [Данные передаются в HybridRecommender] ->

Параллельные действия:

[Content-Based фильтр: поиск курсов по схожести с пройденными] -> (A)

[Collab-Based фильтр: поиск курсов через SVD модель] -> (A)

(A) -> [Объединение и ранжирование результатов от двух фильтров] ->

[Возврат топ-5 рекомендаций] -> [Отображение рекомендаций студенту] -> (Конец)

7. Описание интерфейсов и API

Внутренний интерфейс: Класс RecommendationEngine имеет метод get_hybrid_rec(student_id, n=5), который возвращает список ID рекомендованных курсов.

Внешний API (для будущего развития):

GET /api/student/{id}/recommendations - получить рекомендации для студента.

GET /api/courses - получить каталог курсов.

POST /api/admin/courses - добавить новый курс (для деканата).

8. Разработка прототипа

Бэкенд: Python, Flask/FastAPI.

Интеллектуальный модуль: Библиотеки pandas, numpy, scikit-surprise, scikit-learn.

Фронтенд (базовый): Шаблонизатор Jinja2 или простой React-интерфейс.

База данных: SQLite для прототипа.

Функционал прототипа:

Страница входа/профиля студента.

Страница "Каталог курсов".

Страница "Мои рекомендации" с списком из 5 персонально рекомендованных курсов и кратким объяснением ("Потому что вы хорошо успеваете по Python и Data Science").

Админ-панель для деканата (добавление курсов с тегами).

9. Тестирование и демонстрация

Тестирование:

Модульный *тест:* Проверка, что метод `get_content_based_rec` возвращает непустой список для студента с историей.

Интеграционный *тест:* Эндпоинт `/recommendations` возвращает корректный JSON с 5 курсами.

Сценарий демонстрации:

1. "Показываем профиль студента Петрова, который успешно сдал 'Введение в Python' и 'Математический анализ'".
2. "Переходим на вкладку 'Рекомендации'. Система предлагает курсы 'Машинное обучение', 'Алгоритмы и структуры данных', 'Статистика'".
3. "Объясняем: эти курсы рекомендованы, потому что они логично продолжают пройденные Петровым дисциплины и популярны среди студентов с похожим профилем".

Отчет

Титульный лист.

Оглавление.

Введение (Актуальность, цель, задачи).

Глава 1. Анализ предметной области и проектирование интеллектуальной системы рекомендаций.

Глава 2. Выбор и реализация гибридного алгоритма рекомендаций.

Глава 3. Проектирование и разработка программного прототипа "IntelliCourse" (с UML-диаграммами).

Глава 4. Тестирование и результаты работы системы.

Заключение (Выводы, перспективы).

Список литературы.

Приложения (Исходный код, примеры синтетических данных).

Шаблон презентации для защиты (10-12 слайдов)

- **Слайд 1: Титульный.** IntelliCourse. Интеллектуальная система рекомендации курсов.
- **Слайд 2: Проблема.** "Студенты теряются в выборе курсов -> неоптимальная траектория -> снижение мотивации".
- **Слайд 3: Решение и Цель.** Представить систему IntelliCourse. Цель: разработка персонализированного рекомендателя.
- **Слайд 4: Как это работает?** Схема: Данные студента + Каталог курсов -> Гибридная модель -> Список рекомендаций.

- **Слайд 5: Выбор технологии.** Гибридная модель: SVD (для похожих студентов) + Content-Based (для похожих курсов). *Графическое сравнение подходов.*
- **Слайд 6: Результаты обучения модели.** *Скриншот из Jupyter Notebook с метриками:* RMSE=0.85, Precision@5=0.65.
- **Слайд 7: Архитектура системы.** *Диаграмма классов с выделенным классом RecommendationEngine.*
- **Слайд 8: Прототип системы.** *Скриншот интерфейса студента со списком рекомендаций.*
- **Слайд 9: Демонстрация.** *Живой показ работы прототипа по подготовленному сценарию.*
- **Слайд 10: Выводы.** Система разработана, прототип функционирует, показатели качества рекомендаций являются удовлетворительными.
- **Слайд 11: Что дальше?** Интеграция с реальной АИС ВУЗа, учет данных с hh.ru о востребованности навыков.
- **Слайд 12: Спасибо за внимание! Вопросы?**